

- Correction du TD 9 -

Parcours d'un graphe en profondeur :
DFS (*Depth First Search*)

Partie 1 : Retour sur le parcours BFS du Devoir Surveillé

Le tableau A est associé à la fonction n° 2.

Le tableau B est associé à la fonction n° 3.

Le tableau C est associé à la fonction n° 4.

Le tableau D est associé à la fonction n° 1.

Partie 2 : Algorithme DFS

Exercice 1 :

1. Les listes ci-dessous ont été obtenues grâce à la fonction écrite et complétée à la fin de la question 2a.

- Graphe G_1 , sommet source $s = 5$
Fin de la boucle n°1
Access = [5, 3] Avisiter = [5, 3]
Fin de la boucle n°2
Access = [5, 3, 1] Avisiter = [5, 3, 1]
Fin de la boucle n°3
Access = [5, 3, 1, 0] Avisiter = [5, 3, 1, 0]
Fin de la boucle n°4
Access = [5, 3, 1, 0, 2] Avisiter = [5, 3, 1, 0, 2]
Fin de la boucle n°5
Access = [5, 3, 1, 0, 2, 4] Avisiter = [5, 3, 1, 0, 2, 4]
Fin de la boucle n°6
Access = [5, 3, 1, 0, 2, 4] Avisiter = [5, 3, 1, 0, 2]
Fin de la boucle n°7
Access = [5, 3, 1, 0, 2, 4, 6] Avisiter = [5, 3, 1, 0, 2, 6]
Fin de la boucle n°8
Access = [5, 3, 1, 0, 2, 4, 6] Avisiter = [5, 3, 1, 0, 2]
Fin de la boucle n°9
Access = [5, 3, 1, 0, 2, 4, 6] Avisiter = [5, 3, 1, 0]
Fin de la boucle n°10
Access = [5, 3, 1, 0, 2, 4, 6] Avisiter = [5, 3, 1]
Fin de la boucle n°11
Access = [5, 3, 1, 0, 2, 4, 6] Avisiter = [5, 3]
Fin de la boucle n°12
Access = [5, 3, 1, 0, 2, 4, 6] Avisiter = [5]
Fin de la boucle n°13
Access = [5, 3, 1, 0, 2, 4, 6] Avisiter = []

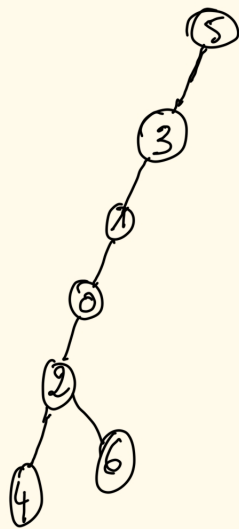
- Graphe G_1 , sommet source $s = 2$
 Fin de la boucle n°1
 Access =[2, 0] Avisiter =[2, 0]
 Fin de la boucle n°2
 Access =[2, 0, 1] Avisiter =[2, 0, 1]
 Fin de la boucle n°3
 Access =[2, 0, 1, 3] Avisiter =[2, 0, 1, 3]
 Fin de la boucle n°4
 Access =[2, 0, 1, 3, 5] Avisiter =[2, 0, 1, 3, 5]
 Fin de la boucle n°5
 Access =[2, 0, 1, 3, 5, 6] Avisiter =[2, 0, 1, 3, 5, 6]
 Fin de la boucle n°6
 Access =[2, 0, 1, 3, 5, 6] Avisiter =[2, 0, 1, 3, 5]
 Fin de la boucle n°7
 Access =[2, 0, 1, 3, 5, 6] Avisiter =[2, 0, 1, 3]
 Fin de la boucle n°8
 Access =[2, 0, 1, 3, 5, 6] Avisiter =[2, 0, 1]
 Fin de la boucle n°9
 Access =[2, 0, 1, 3, 5, 6] Avisiter =[2, 0]
 Fin de la boucle n°10
 Access =[2, 0, 1, 3, 5, 6, 4] Avisiter =[2, 0, 4]
 Fin de la boucle n°11
 Access =[2, 0, 1, 3, 5, 6, 4] Avisiter =[2, 0]
 Fin de la boucle n°12
 Access =[2, 0, 1, 3, 5, 6, 4] Avisiter =[2]
 Fin de la boucle n°13
 Access =[2, 0, 1, 3, 5, 6, 4] Avisiter =[]
- Graphe G_1 , sommet source $s = 1$
 Fin de la boucle n°1
 Access =[1, 0] Avisiter =[1, 0]
 Fin de la boucle n°2
 Access =[1, 0, 2] Avisiter =[1, 0, 2]
 Fin de la boucle n°3
 Access =[1, 0, 2, 3] Avisiter =[1, 0, 2, 3]
 Fin de la boucle n°4
 Access =[1, 0, 2, 3, 5] Avisiter =[1, 0, 2, 3, 5]
 Fin de la boucle n°5
 Access =[1, 0, 2, 3, 5, 6] Avisiter =[1, 0, 2, 3, 5, 6]
 Fin de la boucle n°6
 Access =[1, 0, 2, 3, 5, 6] Avisiter =[1, 0, 2, 3, 5]
 Fin de la boucle n°7
 Access =[1, 0, 2, 3, 5, 6] Avisiter =[1, 0, 2, 3]
 Fin de la boucle n°8
 Access =[1, 0, 2, 3, 5, 6] Avisiter =[1, 0, 2]
 Fin de la boucle n°9

- Access =[1, 0, 2, 3, 5, 6, 4] Avisiter =[1, 0, 2, 4]
 Fin de la boucle n°10
 Access =[1, 0, 2, 3, 5, 6, 4] Avisiter =[1, 0, 2]
 Fin de la boucle n°11
 Access =[1, 0, 2, 3, 5, 6, 4] Avisiter =[1, 0]
 Fin de la boucle n°12
 Access =[1, 0, 2, 3, 5, 6, 4] Avisiter =[1]
 Fin de la boucle n°13
 Access =[1, 0, 2, 3, 5, 6, 4] Avisiter =[]
- Graphe G_2 , sommet source $s = 5$
 Fin de la boucle n°1
 Access =[5, 6] Avisiter =[5, 6]
 Fin de la boucle n°2
 Access =[5, 6] Avisiter =[5]
 Fin de la boucle n°3
 Access =[5, 6] Avisiter =[]
 - Graphe G_2 , sommet source $s = 2$
 Fin de la boucle n°1
 Access =[2, 1] Avisiter =[2, 1]
 Fin de la boucle n°2
 Access =[2, 1, 0] Avisiter =[2, 1, 0]
 Fin de la boucle n°3
 Access =[2, 1, 0, 4] Avisiter =[2, 1, 0, 4]
 Fin de la boucle n°4
 Access =[2, 1, 0, 4] Avisiter =[2, 1, 0]
 Fin de la boucle n°5
 Access =[2, 1, 0, 4] Avisiter =[2, 1]
 Fin de la boucle n°6
 Access =[2, 1, 0, 4, 3] Avisiter =[2, 1, 3]
 Fin de la boucle n°7
 Access =[2, 1, 0, 4, 3, 5] Avisiter =[2, 1, 3, 5]
 Fin de la boucle n°8
 Access =[2, 1, 0, 4, 3, 5, 6] Avisiter =[2, 1, 3, 5, 6]
 Fin de la boucle n°9
 Access =[2, 1, 0, 4, 3, 5, 6] Avisiter =[2, 1, 3, 5]
 Fin de la boucle n°10
 Access =[2, 1, 0, 4, 3, 5, 6] Avisiter =[2, 1, 3]
 Fin de la boucle n°11
 Access =[2, 1, 0, 4, 3, 5, 6] Avisiter =[2, 1]
 Fin de la boucle n°12
 Access =[2, 1, 0, 4, 3, 5, 6] Avisiter =[2]
 Fin de la boucle n°13
 Access =[2, 1, 0, 4, 3, 5, 6] Avisiter =[]
 - Graphe G_2 , sommet source $s = 1$
 Fin de la boucle n°1

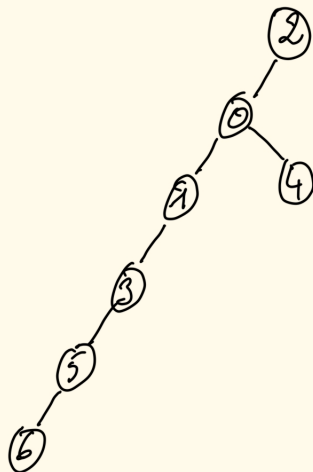
Access =[1, 0] Avisiter =[1, 0]
 Fin de la boucle n°2
 Access =[1, 0, 2] Avisiter =[1, 0, 2]
 Fin de la boucle n°3
 Access =[1, 0, 2, 6] Avisiter =[1, 0, 2, 6]
 Fin de la boucle n°4
 Access =[1, 0, 2, 6] Avisiter =[1, 0, 2]
 Fin de la boucle n°5
 Access =[1, 0, 2, 6] Avisiter =[1, 0]
 Fin de la boucle n°6
 Access =[1, 0, 2, 6, 4] Avisiter =[1, 0, 4]
 Fin de la boucle n°7
 Access =[1, 0, 2, 6, 4] Avisiter =[1, 0]
 Fin de la boucle n°8
 Access =[1, 0, 2, 6, 4] Avisiter =[1]
 Fin de la boucle n°9
 Access =[1, 0, 2, 6, 4, 3] Avisiter =[1, 3]
 Fin de la boucle n°10
 Access =[1, 0, 2, 6, 4, 3, 5] Avisiter =[1, 3, 5]
 Fin de la boucle n°11
 Access =[1, 0, 2, 6, 4, 3, 5] Avisiter =[1, 3]
 Fin de la boucle n°12
 Access =[1, 0, 2, 6, 4, 3, 5] Avisiter =[1]
 Fin de la boucle n°13
 Access =[1, 0, 2, 6, 4, 3, 5] Avisiter =[]

- Représentation des arbres associés au parcours DFS

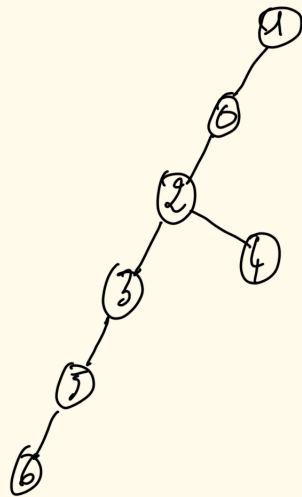
Arbre DFS pour G_1 et $s=5$



Arbre DFS pour G_1 et $s=2$



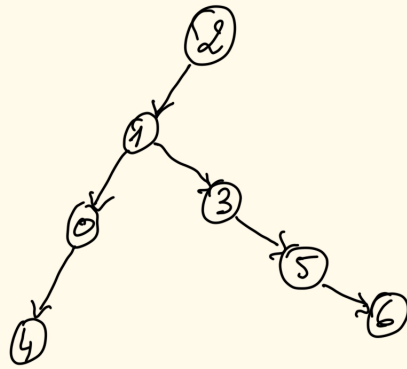
Arbre DFS pour G_1 et $s=1$



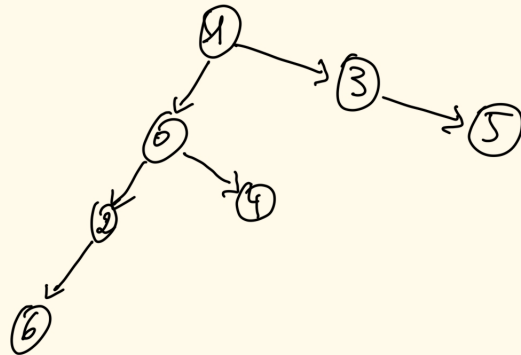
Arbre DFS pour G_2 et $s=5$



Arbre DFS pour G_2 et $s=2$



Arbre DFS pour G_2 et $s=1$



2. (a) Algorithme complété :

```
def ParcoursDFS(G,s):
    Access=[s]
    Avisiter=[s]
    while Avisiter!=[]:
        u=Avisiter.pop()
        L=list(G[u])
        d=len(L)
        k=0
        while k<d:
            x=L[k]
            if not (x in Access):
                Avisiter.append(u)
                Avisiter.append(x)
                Access.append(x)
                k=d
            k=k+1
    return(Access)
```


Ajout de commandes pour obtenir le tableau :

```
def ParcoursDFStableau(G,s):
    Access=[s]
    Avisiter=[s]
    b=1
    while Avisiter!=[]:
        u=Avisiter.pop()
        L=list(G[u])
        d=len(L)
        k=0
        while k<d:
            x=L[k]
            if not (x in Access):
                Avisiter.append(u)
                Avisiter.append(x)
                Access.append(x)
                k=d
            k=k+1
        print('Fin de la boucle n°'+repr(b))
        print(' ')
        print('Access =' +repr(Access)+ '      Avisiter =' +\
              +repr(Avisiter))
        print(' ')
        b=b+1
    return(Access)
```

(b) Algorithme pour une matrice d'adjacences :

```
def ParcoursDFS2(M,s):
    n=len(M)
    Access=[s]
    Avisiter=[s]
    while Avisiter!=[]:
        u=Avisiter.pop()
        k=0
        while k<n:
            if M[u,k]>0 and not (k in Access):
                Avisiter.append(u)
                Avisiter.append(k)
                Access.append(k)
                k=n
            k=k+1
    return(Access)
```

```
def ParcoursDFSrec(G,s,Access):
    Access.append(s)
    for u in G[s]:
        if not(u in Access):
            ParcoursDFSrec(G,u,Access)
    return(Access)
```

sommets voisins d'un sommet étudiés sont parcourus dans le même sens, avec l'algorithme itératif et l'algorithme récursif.

Les listes de parcours obtenues à l'issue de l'algorithme sont identiques.

Exercice 2 :

1.

```
def Est_connexe(G):
    L=list(G.keys())
    s=L[0]
    Acc=ParcoursDFS(G,s)
    n=len(G)
    if len(Acc)==n:
        return('Connexe')
    else:
        return('Non Connexe')
```

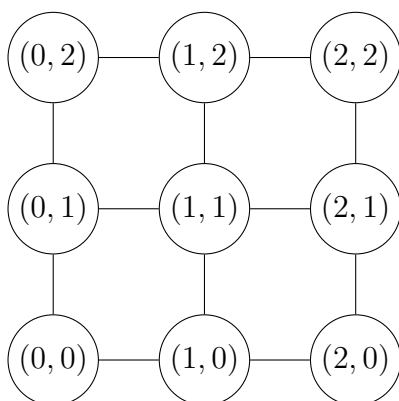
2. (a) Comme G est connexe, tout sommet distinct de s est relié à s par une chaîne. L'algorithme de parcours en profondeur d'un graphe permet d'atteindre tous les sommets à partir du sommet source. D'où $L = S$.
De plus, dans l'algorithme en profondeur, dès qu'un sommet est découvert, il est marqué en gris : on en cherche ses voisins, mais on ne repasse pas sur un sommet gris. Ainsi, le graphe G_s ne contient pas de cycle.
Enfin, le graphe G_s n'est pas orienté.
Finalement, G_s est un arbre.

(b)

```
def arbrecouvrant(M,s):
    n=len(M)
    Access=[s]
    Avisiter=[s]
    Parent=['vide' for k in range(n)]
    while Avisiter!=[]:
        u=Avisiter.pop()
        k=0
        while k<n:
            if M[u,k]>0 and not (k in Access):
                Avisiter.append(u)
                Avisiter.append(k)
                Access.append(k)
                Parent[k]=u
                k=n
            k=k+1
        A=np.zeros([n,n])
    for i in range(n):
        if i!=s:
            j=Parent[i]
            A[i,j]=1
            A[j,i]=1
    return(A)
```

3. A chaque passage dans la boucle **while**, x désigne l'un des sommets du graphe G « qui n'a pas encore été traité » et on en détermine la composante fortement connexe dex : on cherche tous les sommets $y_1, \dots, y_k$ atteignables depuis le sommet x , puis pour chacun de ces sommets y_i , on vérifie si on peut atteindre x depuis y_i .
Après exécution, la fonction **Mystere** renvoie le nombre de composantes fortement connexe du graphe G (associé au dictionnaire G de la fonction Python).

Exercice 3 :



1.

```

def Grille(n):
    G = {}
    for x in range(0, n):
        for y in range(0, n):
            voisins = []
            for (vx, vy) in [(x+1,y), (x, y+1), \
                            (x-1,y), (x, y-1)]:
                if 0 <= vx and vx<n and 0 <= vy and vy < n:
                    voisins.append((vx,vy))
            G[(x,y)] = voisins
    return G

```

3. (a) Par exemple, pour $(0,0)$ comme point de départ, les deux fonctions Python renvoient respectivement :

Avec PP1 et $n=2$: $([0, 1, 1, 0], [0, 0, 1, 1])$

Avec PP2 et $n=2$: $([0, 0, 1, 1], [0, 1, 1, 0])$

Autre exemple :

Avec PP1 et $n=5$:

$([0, 1, 2, 3, 4, 4, 4, 4, 4, 3, 2, 1, 0, 0, 1, 2, 3, 3, 2, 1, 0, 0, 1, 2, 3],$
 $[0, 0, 0, 0, 0, 1, 2, 3, 4, 4, 4, 4, 4, 3, 3, 3, 3, 2, 2, 2, 2, 1, 1, 1, 1])$

Avec PP2 et $n=5$:

$([0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4],$
 $[0, 1, 2, 3, 4, 4, 3, 2, 1, 0, 0, 1, 2, 3, 4, 4, 3, 2, 1, 0, 0, 1, 2, 3, 4])$

Les deux résultats ne sont pas identiques.

*Noter que les explications ci-dessous dépendent de la fonction **Grille** écrite à la question précédente : l'ordre de parcours dépend de l'ordre de la liste des voisins d'un sommet (x,y) donnée dans le dictionnaire G .*

En effet, la première fonction reproduit exactement l'algorithme expliqué en cours, avec le graphe défini par la fonction **Grille** donnée dans la question précédente :

→ la liste des voisins du sommet (x,y) est donnée dans l'ordre (les points hors de la grille seront omis/oubliés) : $(x+1,y)$, $(x,y+1)$, $(x-1,y)$ et $(x,y-1)$.

→ Ainsi, pour le parcours en profondeur avec la fonction PP1, on commence par aller « le plus possible à droite ».

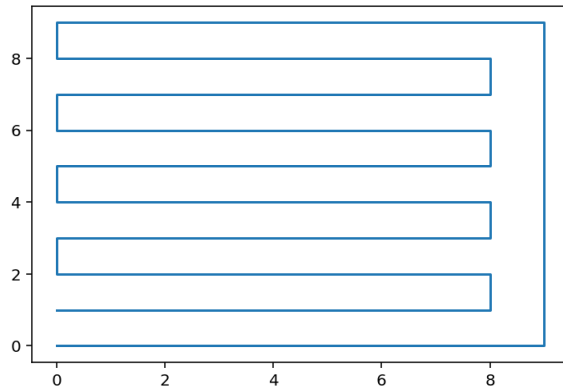
→ Quand on est bloqué, on suit le parcours en allant vers le haut.

→ Quand on est bloqué, on suit le parcours en allant vers la gauche.

→ Quand on est bloqué, on suit le parcours en allant vers le haut.

→ Quand on est bloqué, on descend d'une unité, puis on suit le parcours vers la droite...

Exemple de parcours en partant de $(0,0)$, pour $n = 10$, avec PP1 :



Avec le parcours PP2, dès que l'on découvre un sommet, on garde en mémoire tous ces sommets (dans la pile, sans les noter comme « découverts »).

Avec le parcours PP2, d'un sommet (a, b) , on se déplace sur le dernier (dans la liste des voisins) sommet voisin noté blanc de (a, b) .

Ainsi, avec le parcours PP2,

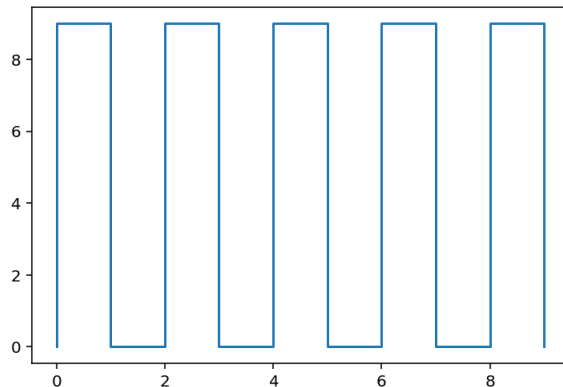
→ on part de $(0, 0)$,

→ on se déplace sur $(0, 1)$, puis on continue de monter (les derniers voisins du sommet considérés sont soit hors de la grille, soit gris),

→ puis quand on est bloqué, on dépile le haut de la pile, et on visite de dernier sommet rencontré non encore étudié : il s'agit du sommet $(1, n - 1)$,

→ on mémorise les voisins de $(1, n - 1)$ et on se déplace sur le dernier mémorisé et blanc : on redescend...

Exemple de parcours en partant de $(0, 0)$, pour $n = 10$, avec PP2 :



(b) Exemple de fonction :

```

def PL(n, depart):
    G=Grille(n)
    file = [depart]
    marquage = {}
    for (x,y) in G.keys():
        marquage[(x,y)] = 'Blanc'
    liste_x, liste_y = [], []
    while len(file) > 0:
        (x, y) = file.pop()
        if marquage[(x,y)]=='Blanc':
            marquage[(x,y)] = 'Gris'
            liste_x.append(x)
            liste_y.append(y)
            for voisin in G[(x,y)]:
                file.insert(0,voisin)
    return (liste_x, liste_y)

```

4. (a)

```

def afficherPP1(n, depart):
    liste_x, liste_y = PP1(n, depart)
    liste_c = [i for i in range(n**2)]
    plt.scatter(liste_x, liste_y, c=liste_c)
    return

```

(b)

```

def afficherPL(n, depart):
    liste_x, liste_y = PL(n, depart)
    liste_c = [i for i in range(n**2)]
    plt.scatter(liste_x, liste_y, c=liste_c)
    return

```

(c) Observer!

(d) Cette différence (ou non!) est expliquée dans la question 3a : si vous et votre voisin n'ont pas ordonné la liste des voisins d'un sommet dans la fonction Python **Grille** définissant le graphe G_n , les graphiques obtenus ne sont pas identiques.
